CampusLoop Web Application Development Specification

A practical blueprint for designing, implementing, testing, and deploying the CampusLoop web app

Project: CampusLoop — Smart Campus Resource Sharing
Recommended implementation: React + Node.js/Express + MongoDB + Firebase Authentication + Cloud Storage
Development style: Build a small working MVP first, then add advanced features

> **Important:** This document distinguishes required MVP features from optional features. Do not build every idea at once. First complete listing, search, requests, approval, QR handover, QR return, history, and admin control.

1. Product Vision

CampusLoop helps verified students discover and circulate underused academic and campus resources. A student can list an item, another student can request it, the owner can approve the request, both parties can verify handover and return through QR, and the system can record reuse history and award Green Points.

MVP success condition

> A verified student must be able to complete the entire flow using a real test item: register → list → search → request → approve → QR handover → borrow → QR return → history update.

2. User Roles

Role	Responsibilities	Access
Student / owner	List items, approve or reject requests, confirm handover, confirm returns, report disputes.	Own listings, received requests, active transactions, history.
Student / borrower	Search items, send requests, scan QR, confirm condition, return items, view history.	Marketplace, own requests, active borrowings, history.
Administrator	Verify users, moderate listings, manage categories, resolve disputes, view reports, suspend accounts.	Admin dashboard and system-wide records.

3. Recommended Technology Stack

Layer	Recommended technology	Reason
Frontend	React + Vite + TypeScript	Fast component-based development and maintainable code.
Styling	Tailwind CSS or CSS modules	Responsive UI with consistent design tokens.
Routing	React Router	Separate public, student, and admin routes.
Forms	React Hook Form + Zod	Client-side form handling and validation.
Data fetching	TanStack Query or a clean API service layer	Loading, caching, mutation, and error states.
Backend	Node.js + Express + TypeScript	REST APIs, authorization, and transaction logic.
Database	MongoDB with Mongoose	Flexible item and transaction documents; PostgreSQL is also acceptable.
Authentication	Firebase Authentication	Email/password, Google login, and session management.
Images	Cloudinary, Firebase Storage, or S3-compatible storage	Item photographs and condition evidence.

Layer	Recommended technology	Reason
QR	Server-generated signed or random token	Connects the physical item or approved transaction to the backend.
Notifications	Email first; push notifications later	Keep MVP implementation manageable.
Deployment	Vercel/Netlify frontend + Render/Railway/Fly.io backend + managed database	Low-cost student-friendly deployment.

Use one consistent stack for the MVP. Do not mix Firebase Firestore, MongoDB, and another database unless your team has a clear architectural reason.

4. Application Modules

Module	MVP requirement	Priority
Authentication	Registration, login, logout, email verification, role checks, profile.	Must have
User profile	Name, department, semester, verification state, rating/reputation summary.	Must have
Item management	Create, edit, delete, pause, and view listings with photos and condition.	Must have
Marketplace	Search, category filter, availability filter, item detail page.	Must have
Requests	Send, approve, reject, cancel, and view request status.	Must have
Transactions	Due date, handover, active borrowing, return, completion, overdue state.	Must have
QR verification	QR generation, scan, backend validation, handover and return confirmation.	Must have
Notifications	Request received, approval, rejection, due reminder, return confirmation.	Should have
History	Completed and active transactions for owner and borrower.	Must have
Rewards	Green Points for completed reuse actions.	Should have
Admin	User, item, request, dispute, category, and report management.	Must have
Disputes	Damage, missing item, late return, inappropriate listing report.	Should have
Chat	Direct messaging between owner and borrower.	Later
AI recommendations	Suggested items or AI-generated descriptions.	Later

5. Route Map

Route	Page	Access
/	Landing page	Public
/login	Login	Public
/register	Registration	Public
/verify-email	Email verification	Authenticated
/dashboard	Student dashboard	Student
/items	Browse marketplace	Student
/items/new	Create listing	Student
/items/:itemId	Item details	Student
/items/:itemId/edit	Edit listing	Owner

Route	Page	Access
/requests	Requests sent and received	Student
/transactions	Active and completed transactions	Student
/transactions/:transactionId	Transaction details and QR actions	Owner or borrower
/history	Borrowing and lending history	Student
/profile	Profile and reputation	Student
/notifications	Notifications	Student
/admin	Admin dashboard	Admin
/admin/users	User management	Admin
/admin/items	Item moderation	Admin
/admin/disputes	Dispute management	Admin
/admin/analytics	Usage and impact analytics	Admin

6. Core User Flows

6.1 Registration and verification

1. User submits name, college email, department, semester, and password.
2. Backend creates a user with status PENDING or VERIFIED according to the chosen verification method.
3. Email verification is completed.
4. Backend assigns STUDENT role by default; ADMIN must be assigned manually.
5. User can access student features only after verification.

6.2 Create item listing

1. User opens “List an Item.”
2. Adds title, description, category, photos, condition, action type, location, availability, and maximum borrowing duration.
3. Frontend validates required fields and image size/type.
4. Backend stores the item with status AVAILABLE or PENDING_REVIEW.
5. System generates a secure QR token after the item is accepted.

6.3 Borrow request

1. Borrower opens an available item.
2. Selects dates and submits a message or purpose.
3. Backend confirms that the borrower is verified, is not the owner, and has no overlapping active transaction.
4. Request is stored as PENDING.
5. Owner receives a notification.

6.4 Owner approval

1. Owner reviews the request and borrower profile.
2. Owner accepts or rejects.
3. If accepted, backend checks that the item is still available for the selected dates.
4. Request becomes APPROVED and item becomes RESERVED.
5. Other overlapping pending requests are rejected or marked unavailable.

7. QR Verification Implementation

7.1 Recommended QR design

Do not put private user information or a plain database record in the QR. Generate a random or signed token on the server. The QR can encode a URL such as `https://your-domain.com/verify/transaction-token`. The token must be validated by the backend before any state change.

Recommended approach

• For each item, create a stable item QR for identification.

• For each approved borrowing, create a transaction-specific QR or signed verification token.

• Require the scanner to be authenticated.

• Allow only valid state transitions.

• Use a one-time handover token or mark the token as used after successful handover.

• Use a separate return event or securely regenerate the return token.

• Record actor, timestamp, transaction ID, device/session information where appropriate.

7.2 Handover state flow

```
APPROVED → HANDOVER_PENDING → BORROWED
```

Handover API logic: authenticated user scans token → backend finds token → validates approved transaction → confirms actor is owner or borrower → checks date/time and current state → records condition confirmation → writes handover event → updates item and transaction atomically.

7.3 Return state flow

```
BORROWED → RETURN_PENDING → COMPLETED → AVAILABLE
```

Return API logic: authenticated user scans return token → backend finds active transaction → validates actor and state → records return condition and timestamp → receives dual confirmation if required → closes transaction → marks item available → awards points.

7.4 QR security checklist

• Never trust item status from the frontend.

• Never allow the frontend alone to mark a transaction as returned.

• Expire tokens where practical.

• Prevent duplicate scans and replay attacks.

• Log failed scans and suspicious activity.

• Use authorization middleware on every transaction endpoint.

• Do not reveal another user's personal data through a QR URL.

8. Database Design

Collection/table	Core fields	Notes
users	_id, authUid, name, email, department, semester, role, verificationStatus, reputationScore, points, createdAt	Email and auth UID should be unique.
items	_id, ownerId, title, description, categoryId, images, condition, actionTypes, status, location, availability, maxDuration, qrTokenHash, createdAt	Store token hash rather than raw secret where possible.
requests	_id, itemId, ownerId, borrowerId, startDate, dueDate, purpose, status, createdAt, decidedAt	Use indexes for item/date/status queries.
transactions	_id, requestId, itemId, ownerId, borrowerId, status, handoverEvent, returnEvent, dueDate, createdAt, completedAt	One active transaction per item and time interval.

Collection/table	Core fields	Notes
events	_id, transactionId, type, actorId, timestamp, metadata	Useful for audit trail: REQUESTED, APPROVED, HANDOVER, RETURNED, DISPUTE.
notifications	_id, recipientId, type, title, message, relatedId, readAt, createdAt	Support in-app first; email later.
rewards	_id, userId, transactionId, points, reason, createdAt	Avoid awarding points twice for one event.
disputes	_id, transactionId, reporterId, type, description, evidence, status, resolution, resolvedBy	Admin review required.
categories	_id, name, active, createdAt	Admin-managed.

9. API Specification

Method	Endpoint	Purpose	Auth
POST	/api/auth/profile	Create or update application profile after authentication.	Student
GET	/api/items	List available items with filters and pagination.	Student
POST	/api/items	Create listing.	Student
GET	/api/items/:id	View item details.	Student
PATCH	/api/items/:id	Edit item owned by current user.	Owner
DELETE	/api/items/:id	Archive or remove item if no active transaction.	Owner/Admin
POST	/api/items/:id/requests	Create borrow request.	Student
GET	/api/requests/received	Owner’s received requests.	Student
PATCH	/api/requests/:id/decision	Accept or reject request.	Owner
GET	/api/transactions	List current user transactions.	Student
POST	/api/transactions/:id/handover/verify	Validate handover QR and create event.	Owner/Borrower
POST	/api/transactions/:id/return/verify	Validate return QR and create event.	Owner/Borrower
POST	/api/transactions/:id/confirm	Confirm condition or receipt.	Owner/Borrower
POST	/api/disputes	Open a dispute.	Student
GET	/api/admin/analytics	Dashboard metrics.	Admin

10. Backend Rules and Authorization

• A user cannot request their own item.

• Only the owner can accept or reject a request.

• Only the owner or approved borrower can access transaction actions.

• An item cannot have overlapping approved active transactions.

• An item cannot be edited in ways that change its identity during active borrowing.

• A request can be cancelled only according to defined status rules.

• Only a valid approved transaction can move to BORROWED.

• Only an active BORROWED transaction can move to RETURN_PENDING or COMPLETED.

• Green Points are awarded only once after a completed transaction.

• Admins can suspend users, hide listings, and resolve disputes.

11. Frontend Screen Requirements

Screen	Required UI elements
Landing page	Project purpose, how it works, categories, sustainability message, login/register buttons.
Dashboard	Active borrowing, pending requests, items listed, points, reminders, quick actions.
Marketplace	Search bar, category filter, availability filter, cards, item condition, owner rating.
Item detail	Images, description, condition, owner summary, availability, action buttons, report option.
Create listing	Validated form, image upload, condition selector, action type, availability, duration, location.
Requests	Tabs for sent/received; accept, reject, cancel, view details.
Transaction page	Status timeline, due date, QR action, condition evidence, return action, report dispute.
History	Borrowing/lending tabs, completed status, dates, points earned.
Admin dashboard	Counts, recent activity, pending moderation, disputes, user/item controls, analytics.

12. Development Plan

Sprint	Work	Deliverable
Sprint 0	Confirm requirements, wireframes, database schema, Git workflow, environment variables.	Approved scope and technical design.
Sprint 1	Project setup, authentication, roles, layout, protected routes.	Users can register, log in, and reach dashboards.
Sprint 2	Item model, create/edit listing, image upload, marketplace search.	Working item marketplace.
Sprint 3	Borrow requests, owner approval, status transitions, notifications.	Working request lifecycle.
Sprint 4	Transaction model, due dates, active borrowing, history.	Working transaction lifecycle without QR.
Sprint 5	QR generation, scan page, server validation, handover and return events.	End-to-end QR workflow.
Sprint 6	Admin dashboard, reports, disputes, points, overdue reminders.	MVP management and trust features.
Sprint 7	Testing, security review, usability test, bug fixes, deployment.	Demonstrable release and evidence.

13. Git and Team Workflow

• Use a shared Git repository with main, develop, and feature branches.

• Create issues for each feature and bug.

• Use pull requests and require at least one teammate review.

• Keep commits small and descriptive: feat(items): add listing form.

• Store secrets only in environment variables, never in Git.

• Maintain a README with setup, environment variables, API routes, and deployment instructions.

• Assign one owner for frontend, one for backend/database, one for testing/documentation; rotate review responsibility.

Suggested repository structure

```
campusloop/
■■■ client/
■ ■■■ src/components/
■ ■■■ src/pages/
■ ■■■ src/features/
■ ■■■ src/services/
■ ■■■ src/routes/
■■■ server/
■ ■■■ src/controllers/
■ ■■■ src/models/
■ ■■■ src/routes/
■ ■■■ src/middleware/
■ ■■■ src/services/
■ ■■■ src/utils/
■■■ docs/
■■■ .env.example
■■■ README.md
```

14. Testing Plan

Test area	Example test	Expected result
Authentication	Unverified user attempts to create a listing.	Request blocked with clear message.
Authorization	Borrower attempts to approve another owner’s request.	403/authorization error.
Listing	Submit form without title or condition.	Validation prevents submission.
Availability	Two borrowers request the same date range.	Only one request can be approved.
QR handover	Scan expired, invalid, duplicate, or unauthorized token.	No status change; event logged.
QR handover	Scan valid approved transaction.	Transaction becomes BORROWED once.
QR return	Attempt return before handover.	Operation blocked.
QR return	Valid active transaction is returned.	Transaction completed and item available.
Overdue	Due date passes without return.	Status or reminder changes according to policy.
Dispute	Borrower reports damage after return.	Dispute is created for admin review.
Security	Upload unsupported file type or excessive file size.	Upload rejected safely.
Performance	Marketplace request with pagination and filters.	Acceptable response time under test load.

15. Definition of Done

■ Feature works on desktop and mobile layouts.

■ Loading, empty, success, and error states are implemented.

■ Frontend and backend validation both exist.

■ Authorization rules are tested.

■ Database changes are documented.

■ No secrets are committed.

■ At least one teammate reviews the code.

■ A screenshot or test result is added to documentation.

■ Feature is deployed or reproducible using README instructions.

16. Security Checklist

• Use HTTPS in production.

• Validate and sanitize all input on the server.

• Use secure authentication sessions or verified Firebase tokens.

• Apply role-based access control.

• Use rate limiting for authentication and QR endpoints.

• Restrict CORS to the deployed frontend origin.

• Validate image MIME type, extension, and size; store files outside the application server when possible.

• Do not expose database credentials or service-account keys.

• Use audit logs for handover, return, admin, and dispute events.

• Avoid unnecessary collection of phone numbers, precise location, or identity documents.

• Provide account/report/block mechanisms and clear dispute policy.

17. Deployment Checklist

■ Create production database and restricted database user.

■ Configure frontend and backend environment variables.

■ Set allowed frontend origin in backend CORS.

■ Configure authentication authorized domains.

■ Configure storage security rules.

■ Run database indexes and migrations/seed categories.

■ Deploy backend before frontend API configuration.

■ Test login, listing, request, QR handover, QR return, and admin routes in production.

■ Enable logs and error monitoring.

■ Create a demo account and demo items for mentor presentation.

■ Back up database and document rollback steps.

18. Demo Script for Mentor

• Log in as Owner A.

• Create a calculator or Arduino kit listing with condition and availability.

• Log in as Student B in another browser or incognito window.

• Search and open the listing.

• Submit a borrow request.

• Return to Owner A and approve the request.

• Open the transaction page and show the status timeline.

• Scan or simulate the QR handover using the deployed scan page.

• Show the transaction changing to BORROWED.

• Return to Student B and initiate return.

• Scan the return QR and confirm condition.

• Show the item changing to AVAILABLE and history/points updating.

• Open admin dashboard and show the transaction audit trail.

19. MVP Decision Log

Decision	Chosen option	Reason / mentor approval
Initial item categories	[calculator / lab kit / books / mixed]	[fill]
Verification method	[college email / admin approval / student ID]	[fill]
Database	[MongoDB / PostgreSQL]	[fill]
QR model	[item QR / transaction QR / both]	[fill]
Return confirmation	[owner only / borrower + owner]	[fill]
Condition evidence	[text / photos / checklist]	[fill]
Rewards	[Green Points / none in MVP]	[fill]
Notifications	[in-app / email / push]	[fill]
Deployment	[platforms and URLs]	[fill]

20. Build Order Summary

> Build in this exact order: authentication → roles → item listing → marketplace → borrow request → owner approval → transaction status → QR handover → QR return → history → notifications → admin → rewards → disputes → advanced features.

Do not start with AI recommendations, chat, smart lockers, payments, or a complex reward economy. Those features are useful only after the core transaction lifecycle is reliable.

21. Project Documentation to Maintain

• README.md with setup and deployment instructions.

• Product requirements document.

• Database schema and relationship diagram.

• API endpoint document.

• User-flow and QR sequence diagrams.

• Threat model and security checklist.

• Test-case spreadsheet and results.

• Survey questionnaire and anonymized results.

• Screenshots of each completed module.

• Changelog and known limitations.

22. Final Implementation Position

CampusLoop should be implemented and evaluated as a practical integrated system, not presented as a claim that no similar project exists. Your technical contribution is the working combination of campus verification, student-owned resource listings, borrowing and return states, QR-backed physical transaction events, history, notifications, and incentives. The most important demonstration is a reliable end-to-end transaction with correct backend validation and a clear audit trail.